

[CS230] Improving Deep Neural Networks: Dropout Regularization

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-4. Neural Networks Basics - *Completed*

Chapter 5. Practical Aspects of Deep Learning (Current Unit)

- 5.1 Train / Dev / Test Sets Strategy - *Completed*
- 5.2 Bias vs Variance Analysis - *Completed*
- 5.3 Regularization (L1/L2) - *Completed*
- **5.4 Dropout Regularization**
 - * The Concept: Killing Neurons
 - * Why does it work? (Ensemble Effect)
 - * Inverted Dropout (Scaling)
 - * Implementation Details
- 5.5 Input Normalization - *Upcoming*

지난 시간 복습 및 연결

지난 시간에 우리는 가중치(W)의 크기를 강제로 줄여버리는 **L2 정규화(Weight Decay)**를 배웠습니다. 오늘 배울 기법은 조금 더 '과격'합니다. 모델의 과대적합(Overfitting)을 막기 위해, 학습 과정에서 멀쩡한 뉴런들을 무작위로 '**제거(Kill)**'해버립니다. 바로 **드롭아웃(Dropout)**입니다. "뇌세포를 죽이는데 뇌가 더 똑똑해진다니?"라는 의문이 들겠지만, 이것이 현대 딥러닝에서 가장 강력한 정규화 기법입니다. 그 역설적인 원리를 파헤쳐 보겠습니다.

1 Unit Overview

□ 핵심 목표

이 단원은 신경망의 강건함(Robustness)을 높이는 드롭아웃의 원리와 구현을 다룹니다.

- **원리:** 드롭아웃이 어떻게 특정 뉴런에 대한 의존도(Co-adaptation)를 낮추는지 이해합니다.
- **수학:** 학습과 테스트 시의 출력값 차이를 보정하는 **Inverted Dropout** 기술을 익힙니다.
- **규칙:** 드롭아웃은 오직 **학습(Training)** 때만 켜고, 테스트(Test) 때는 끄는 원칙을 명심합니다.
- **구현:** NumPy를 사용하여 마스크 행렬(Mask Matrix)을 만들고 적용해봅니다.

2 Essential Terminology

용어	변수	설명
Dropout	-	학습 시 뉴런을 무작위로 삭제(0으로 설정)하는 기법.
Keep Probability	keep_prob	뉴런을 '살려둘' 확률. (예: 0.8 = 20% 삭제).
Inverted Dropout	-	학습 시 값을 keep_prob로 나누어 스케일을 보정하는 표준 방식.
Ensemble	-	여러 모델의 예측을 평균 내는 것. 드롭아웃은 이 효과를 냄.

3 Core Concepts: 무작위 삭제의 미학

3.1 1. 직관적 해석 (Why does it work?)

□ 천재에게 의존하는 팀 프로젝트 (직관적 비유)

- **상황 (No Dropout):** 팀에 천재 한 명(특정 뉴런)이 있습니다. 다른 팀원들은 그 천재만 믿고 일을 안 합니다. 만약 천재가 결근하면(새로운 데이터), 프로젝트는 망합니다. (과대적합)
- **상황 (Dropout):** 매일 무작위로 팀원을 출근시키지 않습니다. 천재가 결근할 수도 있습니다.
- **결과:** 팀원들은 누구에게도 의존할 수 없으므로, 모두가 업무 전반을 익히게 됩니다. 결국 팀 전체가 강력하고 유연해집니다.

드롭아웃을 적용하면 뉴런들이 특정 입력(친구)에만 의존하지 않고, 가중치를 골고루 분산(Spread out)시키게 됩니다. 이는 L2 정규화와 비슷한 효과를 냅니다.

4 Deep Dive: Inverted Dropout (역 드롭아웃)

이 섹션은 면접 단골 질문인 "왜 학습 때 값을 나누나요?"에 대한 답입니다.

□ The Scale Problem (수학적 원리)

keep_prob = 0.5 (50% 삭제)라고 가정합니다.

1. 학습 단계 (Train): 뉴런의 절반이 0이 되므로, 다음 층으로 전달되는 합계 ($Z = \sum w_i a_i$)도 대략 절반으로 줄어듭니다.

2. 테스트 단계 (Test): 테스트 때는 드롭아웃을 끕니다(모든 뉴런 사용). Z 값이 학습 때보다 2배 뻗튀기 됩니다. 예측값이 완전히 달라집니다.

3. 해결책 (Inverted Dropout): 학습 단계에서 살아남은 뉴런의 값을 미리 2배로 키워줍니다 ($A /= 0.5$). 이렇게 하면 학습 때의 기댓값 ($E[A]$)이 테스트 때와 비슷하게 유지됩니다. 테스트 때는 아무런 연산도 할 필요가 없어집니다.

5 Implementation: Dropout Layer

가장 중요한 것은 'is_training' . . .

```
1 import numpy as np
2
3 class Dropout:
4     def __init__(self, keep_prob=0.8):
5         self.keep_prob = keep_prob
6         self.mask = None # 역전파용 마스크 저장
7
8     def forward(self, A, is_training=True):
9         """
10        A: Activation values
11        """
12        if is_training:
13            # 1. 마스크 생성 (0 ~ 1 난수 < keep_prob)
14            # 보다 keep_prob 작으면 True(1), 크면 False(0)
15            D = np.random.rand(A.shape[0], A.shape[1])
16            D = (D < self.keep_prob).astype(int)
17            self.mask = D
18
19            # 2. 뉴런 끄기 (Shut down)
20            A = A * D
21
22            # 3. 스케일 보정 (Inverted Dropout 핵심!)
23            A = A / self.keep_prob
24
25        return A
26
27     def backward(self, dA):
28         """
29        역전파: 죽은 뉴런은 미분값도 이어야 함
30        """
31        # 1. 마스크 적용
32        dA = dA * self.mask
33
34        # 2. 스케일 보정 순전파( 때 나뉘었으니 여기서도 나뉘어야 함)
35        dA = dA / self.keep_prob
36
37        return dA
38
39 # --- 실행 예제 ---
40 if __name__ == "__main__":
41     np.random.seed(1)
42     A = np.ones((5, 3)) * 10 # 모든 값이 인 10 행렬
43
44     dropout = Dropout(keep_prob=0.8)
45
```

```

46 # Train Mode
47 A_train = dropout.forward(A, is_training=True)
48 print("Train Output:\n", A_train)
49 # 일부는 0, 나머지는 12.5 (10 / 0.8)가 됨
50
51 # Test Mode
52 A_test = dropout.forward(A, is_training=False)
53 print("\nTest Output:\n", A_test)
54 # 원본 그대로 10 유지

```

Listing 1: Inverted Dropout Implementation

6 FAQ & Pitfalls

비용 함수(Cost Function) 진동 문제 (오해 방지 가이드)

드롭아웃을 쓰면 매번 네트워크 구조가 무작위로 바뀝니다. 따라서 비용 함수 J 가 매끄럽게 내려가지 않고 툼니바퀴처럼 진동할 수 있습니다. 디버깅할 때는 잠시 `keep_prob = 1.0`로 J 를 고정시켜주세요.

Q. 입력층(Input Layer)에도 드롭아웃을 쓰나요?

A. 보통은 안 씁니다. 원본 데이터(X)를 지워버리면 정보 손실이 너무 크기 때문입니다. 주로 파라미터가 많은 은닉층(FC Layer)에 사용합니다.

Q. keep_prob는 어떻게 정하나요?

A. 과대적합이 심할 것 같은 층(뉴런이 많은 층)은 낮게(0.5), 그렇지 않은 층은 높게(0.8 ~ 1.0) 설정합니다.

다음 단계 (Next Step)

이제 우리는 과대적합을 막는 두 가지 강력한 방패(L2, Dropout)를 얻었습니다. 하지만 모델 학습이 너무 느리다면 어떨까요? 아무리 좋은 모델도 학습에 1년이 걸린다면 무용지물입니다.

다음 시간에는 학습 속도를 비약적으로 높여주는 **[Optimization]** 기술로 넘어갑니다. 그 첫 번째 열쇠인 '입력 정규화(Input Normalization)'가 왜 경사 하강법의 속도를 높이는지 기하학적으로 살펴보겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Dropout:** 학습 시 무작위로 뉴런을 끈다. (Ensemble 효과, 과대적합 방지)
2. **Inverted Dropout:** 학습 시 출력값을 `keep_prob`로 나눠주어 기댓값을 유지한다.
3. **Test Time:** 테스트 시에는 절대 드롭아웃을 쓰지 않는다.
4. **Caution:** Cost 그래프가 진동할 수 있으니 디버깅 시엔 끄고 확인한다.